QUESTION CONSTRUCTION TO AIAP

NHUT NGUYEN
COLIRO

Chapter 1: Defining the Problem Clearly 4
Concept Explanation 4
Examples 4
Exercises 4
Chapter 2: Breaking Down Complex Tasks 4
Concept Explanation 4
Examples 4
Exercises 5
Chapter 3: Asking for Step-by-Step Guidance 5
Concept Explanation 5
Examples 5
Exercises 5
Chapter 4: Requesting Code Examples 5
Concept Explanation 5
Examples 6
Exercises 6
Chapter 5: Asking for Explanations 6
Concept Explanation 6
Examples 6
Exercises 6
Chapter 6: Seeking Best Practices 7
Concept Explanation 7
Examples 7
Exercises 7
Chapter 7: Inquiring About Debugging Tips 7
Concept Explanation 7
Examples 7
Exercises 7
Chapter 8: Using AI for Code Optimization 8
Concept Explanation 8
Examples 8
Exercises 8
Chapter 9: Asking for Tool and Library Recommendations 8
Concept Explanation 8
Examples 8
Exercises 8
Chapter 10: Seeking Help with Documentation 9

CodiRo
Pro in Code = Pro in Life

Concept Explanation 9
Examples 9
Exercises 9
Chapter 11: Requesting AI-Assisted Learning Resources 9
Concept Explanation 9
Examples 9
Exercises 9
Chapter 12: Simulating Coding Interviews with AI 9
Concept Explanation 10
Examples 10
Exercises 10
Chapter 13: Exploring AI's Creative Capabilities 10
Concept Explanation 10
Examples 10
Exercises 10
Chapter 14: Utilizing AI for Project Management 10
Concept Explanation 10
Examples 11
Exercises 11
Chapter 15: Asking for Feedback on Your Code 11
Concept Explanation 11
Examples 11
Exercises 12
Bonus Information – QUESTION CONSTRUCTION 12
1. Start with the Goal 12
2. Provide Context 12
3. Break Down the Problem 12
4. Request Specific Examples 12
5. Ask for Explanations 12
6. Seek Optimization Advice 13
7. Ask for Debugging Help 13
8. Explore Best Practices 13
9. Request Comparisons 13
10. Ask for Resources 13
11. Simulate Real-World Scenarios 13
12. Use AI for Design Suggestions 13
13. Collaborate with AI on Code Writing 14

CodiRo
Pro in Code = Pro in Life

14. Use AI for Documentation 14
15. Confirm Understanding 14
General Tips for Asking Programming Questions: 14
Examples of Well-Constructed Questions: 14
How to Ask About Programmatic Bugs: 15
More Examples of Well-Constructed Questions: 15
Additional Suggestions for Asking Programming Questions: 17
1. Provide Context and Code 17
2. Explain Expected vs. Actual Behavior 18
3. Include Error Messages and Logs 18
4. Describe What You've Tried 19
5. Ask for Best Practices or Alternatives 19
Additional Examples Based on Customer-Provided Code Issues 19
6. Handling Dependencies and Imports 20
7. Issues with Asynchronous Code 21
8. Debugging UI Issues 21
9. Fixing Logic Errors in Algorithms 22
10. Addressing API Integration Issues 22
11. Troubleshooting Deployment Issues 23
12. Handling Version Control Issues 23
13. Optimizing Code Performance 24
14. Understanding Language-Specific Features 24

CodiRo
Pro in Code - Pro in Life

Chapter 1: Defining the Problem Clearly

Concept Explanation

Defining problems clearly is crucial for effective AI assistance. A well-defined problem statement helps AI systems understand the specific requirements and constraints, leading to more relevant and accurate responses. Ambiguity can cause AI to generate solutions that are off-target or not applicable, wasting time and resources.

Examples

Vague Problem Statement: "I need help with my website."

- **Potential Issues:** This statement is too broad, lacking details about what specifically needs help—design, functionality, or content?

Clear Problem Statement: "I need help integrating a payment gateway into my e-commerce website using Stripe."

- **Benefits:** This provides a specific context (payment gateway) and technology (Stripe), making it easier for AI to provide targeted assistance.

Exercises

1. **Refine Problem Statements:**
   - Vague: "My app isn't working."
   - Refined: "My app crashes when users try to log in using Google authentication. Can you help me debug this?"
2. **Predict AI Responses:**
   - Compare AI responses to both vague and clear problem statements to observe differences in quality and relevance.

CodliRo
Pro in Code - Pro in Life

Chapter 2: Breaking Down Complex Tasks

Concept Explanation

Decomposing complex problems into smaller, manageable tasks allows AI to provide more effective and detailed guidance. Each smaller task can be addressed individually, leading to a structured approach to problem-solving.

Examples

Large Task: Building a web application

- **Decomposed Tasks:**
  1. Setting up the development environment
  2. Designing the database schema

3. Creating user authentication
4. Implementing core features
5. Testing and deploying

AI Assistance: AI can offer step-by-step support for each of these tasks, ensuring comprehensive guidance through the entire process.

Exercises

1. **Break Down a Task:**
   - Complex Task: Developing a machine learning model
   - Decomposed Tasks: Data collection, preprocessing, model selection, training, evaluation
2. **AI Assistance for Components:**
   - Use AI to solve each decomposed part and then integrate these solutions into a complete project.

Chapter 3: Asking for Step-by-Step Guidance

Concept Explanation

Step-by-step guidance is particularly useful for beginners and for learning new concepts. AI can function as a tutor, breaking down complex tasks into understandable stages and guiding users through each one.

Examples

Task: Setting up a Python development environment

CodiRo
Pro in Code - Pro in Life

- **Step-by-Step Guidance:**
  1. Install Python
  2. Set up a virtual environment
  3. Install necessary libraries
  4. Configure IDE

Exercises

1. **Request Guidance:**
   - Ask AI for step-by-step instructions on setting up a project, such as creating a basic web app with Flask.
2. **Follow Instructions:**
   - Implement the provided guidance and test to ensure understanding and correctness.

Chapter 4: Requesting Code Examples

Concept Explanation

Code examples are essential for understanding how to implement specific functionalities. AI can generate code snippets tailored to particular needs, which can be used as a reference or starting point.

Examples

Task: Writing a Python function to sort a list

- **Example Code:**



Exercises

1. **Request Code Examples:**
   - Ask for examples of different sorting algorithms or specific features like user authentication in a web application.
2. **Integrate Examples:**
   - Analyze and incorporate the code snippets into your own projects, understanding how they fit into the overall solution.

CodliRo
Pro in Code - Pro in Life

Chapter 5: Asking for Explanations

Concept Explanation

Understanding code and concepts is crucial for effective learning. AI can provide detailed explanations to clarify complex ideas, improving comprehension and application.

Examples

Concept: How a `for` loop works in Python

- **Explanation:** A `for` loop iterates over a sequence (like a list) and executes a block of code for each item in the sequence.

Exercises

1. **Request Explanations:**
   - Ask AI to explain specific code snippets or concepts, such as the purpose of a decorator in Python.
2. **Discuss Explanations:**
   - Review the explanations in class or with peers to deepen understanding.

Chapter 6: Seeking Best Practices

Concept Explanation

Best practices are established methods that lead to better quality and maintainability in code. AI can guide students to adopt these practices, resulting in cleaner and more efficient code.

Examples

Best Practice: Error handling in Python

- **Recommendation:** Use try-except blocks to handle exceptions gracefully and ensure your code can manage unexpected situations.

Exercises

1. **Ask for Best Practices:**
   - Inquire about best practices for specific coding tasks, like structuring a large codebase or managing dependencies.
2. **Apply Best Practices:**
   - Implement these practices in your own code and evaluate their impact on code quality and maintainability.

CodliRo
Pro in Code - Pro in Life

Chapter 7: Inquiring About Debugging Tips

Concept Explanation

Efficient debugging is key to resolving issues in code. AI can assist by diagnosing problems, providing solutions, and suggesting debugging strategies.

Examples

Bug: Syntax error in Python code

- **AI Assistance:** AI can identify common syntax errors and suggest corrections, like missing colons or incorrect indentation.

Exercises

1. **Debug Code:**
   - Provide buggy code snippets and use AI to identify and fix the issues.
2. **Discuss Debugging Process:**
   - Share findings and discuss the debugging process to learn from each other's experiences.

Chapter 8: Using AI for Code Optimization

Concept Explanation

Code optimization improves performance and efficiency. AI can suggest ways to optimize code, such as refining algorithms or improving resource management.

Examples

Optimization Task: Improving the performance of a sorting algorithm

- **AI Suggestion:** Replace a bubble sort with a more efficient algorithm like quicksort or mergesort.

Exercises

1. **Optimize Code:**
   - Use AI to suggest optimizations for a given piece of code and compare performance before and after.
2. **Evaluate Improvements:**
   - Measure and analyze the impact of optimizations on code efficiency and performance.

CodliRo
Pro in Code - Pro in Life

Chapter 9: Asking for Tool and Library Recommendations

Concept Explanation

Choosing the right tools and libraries can significantly enhance productivity and code quality. AI can recommend tools and libraries based on specific project needs.

Examples

Task: Building a web application

- **AI Recommendation:** Suggest libraries such as Flask for the web framework and SQLAlchemy for ORM.

Exercises

1. **Request Recommendations:**
   - Ask AI for suggestions on tools and libraries for different programming tasks or projects.
2. **Implement Recommendations:**
   - Use the recommended tools and libraries in a project and evaluate their effectiveness.

Chapter 10: Seeking Help with Documentation

Concept Explanation

Good documentation is crucial for maintaining and collaborating on code. AI can assist in generating documentation, including comments, README files, and API documentation.

Examples

Documentation Task: Creating a README file

- **AI Assistance:** Generate a README template that includes sections for project overview, installation instructions, and usage examples.

Exercises

1. **Generate Documentation:**
    - Write code and use AI to help generate comprehensive documentation for the codebase.
2. **Review and Revise:**
    - Evaluate the quality of the generated documentation and make necessary revisions.

Chapter 11: Requesting AI-Assisted Learning Resources

Concept Explanation

Continuous learning is essential for staying updated with programming advancements. AI can recommend resources such as tutorials, books, or courses to support learning.

Examples

Learning Task: Advanced Python techniques

- **AI Recommendation:** Suggest resources like online courses or textbooks on advanced Python programming.

Exercises

1. **Request Learning Resources:**
    - Ask AI for recommendations on resources for specific programming topics or skills.
2. **Apply Learning:**
    - Review the recommended resources and apply new knowledge in a project.

Chapter 12: Simulating Coding Interviews with AI

Concept Explanation

Preparing for coding interviews is vital for job readiness. AI can simulate coding interviews by presenting questions and providing feedback on performance.

Examples

Interview Simulation: Algorithmic problem-solving

- **AI Simulation:** Provide practice questions on algorithms and data structures, and offer feedback on solutions.

Exercises

1. **Simulate an Interview:**
    - Go through a mock coding interview with AI, focusing on problem-solving and coding skills.
2. **Review Performance:**
    - Analyze feedback from the simulation to identify areas for improvement.

Chapter 13: Exploring AI's Creative Capabilities

Concept Explanation

Creativity in problem-solving can lead to innovative solutions. AI can generate creative approaches to coding challenges, offering new perspectives.

CodliRo
Pro in Code - Pro in Life

Examples

Creative Problem: Designing a novel feature for an app

- **AI Suggestion:** Propose unique user interface designs or functionality enhancements that are not immediately obvious.

Exercises

1. **Explore Creative Solutions:**
    - Present a challenging problem and use AI to brainstorm and develop creative solutions.
2. **Evaluate Creativity:**
    - Assess the feasibility and impact of AI-generated creative solutions.

Chapter 14: Utilizing AI for Project Management

Concept Explanation

Efficient project management is crucial for delivering software projects on time and within scope. AI can assist by providing tools for task management, scheduling, and progress tracking. This helps streamline the development process, manage resources effectively, and ensure that project goals are met.

Examples

Project Management Task: Structuring a software development project

- **AI Assistance:** AI can help create a project plan, set up milestones, track deadlines, and manage tasks. For instance, AI might suggest a Gantt chart or a Kanban board for visualizing project progress.

Example:

1. **Project Planning:** AI assists in breaking down the project into phases (e.g., planning, design, implementation, testing, deployment).
2. **Task Management:** AI can help assign tasks to team members, set deadlines, and track progress.

Exercises

1. **Project Structuring Exercise:**
   - **Task:** Develop a project plan for creating a mobile app.
   - **Activity:** Use AI to generate a project plan with key milestones, deadlines, and tasks. Review and adjust the plan as needed.
2. **Progress Tracking Exercise:**
   - **Task:** Implement a task management system for a project.
   - **Activity:** Utilize AI to track the progress of tasks, identify potential delays, and adjust the project plan to keep everything on track.

CodiRo
Pro in Code = Pro in Life

Chapter 15: Asking for Feedback on Your Code

Concept Explanation

Constructive feedback is vital for improving coding skills and ensuring code quality. AI can provide feedback on various aspects of your code, including functionality, readability, and efficiency. This helps developers identify areas for improvement and refine their coding practices.

Examples

Feedback Request: Improving code quality

- **AI Assistance:** AI can review your code and offer suggestions for refactoring, improving readability, and enhancing performance.

Example:

1. **Code Review:** Provide a code snippet to AI and ask for feedback on potential improvements.
2. **Refactoring Suggestions:** AI might suggest more efficient algorithms or better coding practices.

Exercises

1. **Submit Code for Feedback:**
    - **Task:** Write a piece of code and use AI to get feedback.
    - **Activity:** Analyze the AI's feedback, make the suggested improvements, and compare the revised code with the original.
2. **Code Improvement Exercise:**
    - **Task:** Review and refactor a provided code snippet using AI feedback.
    - **Activity:** Implement suggested changes and discuss how they improve the code.

Bonus Information – QUESTION CONSTRUCTION

1. Start with the Goal

- **Example:** "How can I..."
    - **Construct the Question:** Identify what you want to achieve. Clearly state the desired outcome or functionality.
    - **Information:** Example: "How can I create a login system using Flask?"

2. Provide Context

- **Example:** "I'm using... and I need to..."
    - **Construct the Question:** Provide relevant background information about the tools, languages, or frameworks you're using.
    - **Information:** Example: "I'm using Flask and SQLAlchemy, and I need to implement user authentication."

CodiRo
Pro in Code - Pro in Life

3. Break Down the Problem

- **Example:** "What are the steps to...?"
    - **Construct the Question:** Divide the larger task into smaller, manageable components.
    - **Information:** Example: "What are the steps to set up a Flask project, connect it to a database, and create an API endpoint?"

4. Request Specific Examples

- **Example:** "Can you provide an example of...?"
    - **Construct the Question:** Ask for code examples or templates that address a particular issue or functionality.
    - **Information:** Example: "Can you provide an example of a Python function that hashes passwords securely?"

5. Ask for Explanations

- **Example:** "Can you explain how...?"
    - **Construct the Question:** When unclear about how something works, ask for a detailed explanation.

- **Information:** Example: "Can you explain how the `filter` function works in Python with an example?"

6. Seek Optimization Advice

- **Example:** "How can I improve...?"
  - **Construct the Question:** Inquire about ways to enhance your code for better performance or readability.
  - **Information:** Example: "How can I improve the performance of this sorting algorithm in Python?"

7. Ask for Debugging Help

- **Example:** "Why is this not working...?"
  - **Construct the Question:** Describe the issue you're facing and request assistance in identifying and fixing the problem.
  - **Information:** Example: "Why is this loop causing an `IndexError` when I try to iterate through a list in Python?"

8. Explore Best Practices

- **Example:** "What is the best way to...?"
  - **Construct the Question:** Ask for the most efficient or recommended approach to solve a problem.
  - **Information:** Example: "What is the best way to handle exceptions in a Python script?"

9. Request Comparisons

- **Example:** "What's the difference between...?"
  - **Construct the Question:** When choosing between multiple approaches, ask for a comparison.
  - **Information:** Example: "What's the difference between using list comprehensions and the `map` function in Python?"

10. Ask for Resources

- **Example:** "Where can I learn more about...?"
  - **Construct the Question:** Inquire about additional learning resources such as tutorials, books, or courses.
  - **Information:** Example: "Where can I learn more about advanced Python techniques?"

11. Simulate Real-World Scenarios

- **Example:** "Can you simulate...?"
  - **Construct the Question:** Ask AI to simulate real-world scenarios like coding interviews or project requirements.
  - **Information:** Example: "Can you simulate a coding interview with string manipulation questions?"

12. Use AI for Design Suggestions

CodliRo
Pro in Code - Pro in Life

- **Example:** "How should I design...?"
    - **Construct the Question:** Request advice on designing systems, databases, or applications.
    - **Information:** Example: "How should I design a RESTful API for an e-commerce application?"

13. Collaborate with AI on Code Writing

- **Example:** "Help me write...?"
    - **Construct the Question:** Collaborate with AI to write functions, scripts, or components.
    - **Information:** Example: "Help me write a Python function that sends an email with an attachment."

14. Use AI for Documentation

- **Example:** "Can you help me document...?"
    - **Construct the Question:** Request AI assistance in creating documentation or comments for code.
    - **Information:** Example: "Can you help me document this Flask API's endpoints?"

15. Confirm Understanding

- **Example:** "Is this the correct approach to...?"
    - **Construct the Question:** After implementing a solution, ask AI to validate or critique your approach.
    - **Information:** Example: "Is this the correct approach to handling user sessions in a Flask web app?"

CodliRo
Pro in Code - Pro in Life

General Tips for Asking Programming Questions:

1. **Be Specific About the Problem**:
    - Clearly describe what you're trying to achieve.
    - Explain what's not working as expected.
2. **Provide Context**:
    - Include relevant code snippets.
    - Mention the programming language and any libraries or frameworks used.
3. **Show What You've Tried**:
    - Describe the debugging steps you've already taken.
    - Mention any solutions you've attempted and why they didn't work.
4. **Include Error Messages**:
    - Provide any error messages or stack traces you're encountering.

Examples of Well-Constructed Questions:

1. **Debugging a Specific Issue**:
    - **Question**: "I'm working with Python and trying to read a CSV file using the `pandas` library. However, I'm getting a `FileNotFoundError`. Here's the code I'm using: `df = pd.read_csv('data.csv')`. I've verified that `data.csv` is in the same directory as my script. What could be causing this issue?"
    - **Suggestion**: Ensure that the file path is correct and the file exists at that location. Also, check for any typos in the file name.

2. **Understanding Error Messages**:
    - **Question**: "I'm seeing this error message in JavaScript: `TypeError: Cannot read property 'length' of undefined`. It occurs when I try to access the length of an array. Here's the relevant code snippet: `console.log(myArray.length);`. How can I fix this error?"
    - **Suggestion**: Check if `myArray` is properly initialized and not `undefined`. Verify that the variable is assigned correctly before accessing its `length` property.
3. **Clarifying Code Behavior**:
    - **Question**: "In my JavaScript function, the array sorting isn't working as expected. Here's my function: `arr.sort((a, b) => a - b);`. The array is `[5, 2, 9, 1]`, but it's not sorted in ascending order. What might be going wrong?"
    - **Suggestion**: Ensure that `arr` is indeed an array of numbers and that you are calling `sort()` correctly. Verify that there are no side effects modifying the array.
4. **Optimizing Code**:
    - **Question**: "I have a Python function that calculates the factorial of a number using recursion. It works, but it's slow for large inputs. Here's the code: `def factorial(n): return 1 if n == 0 else n * factorial(n-1)`. How can I optimize this function?"
    - **Suggestion**: Consider using an iterative approach or memoization to optimize the factorial calculation.
5. **Integration Issues**:
    - **Question**: "I'm integrating an API with my Node.js application using the `axios` library. My `POST` request is returning a `404 Not Found` error. Here's the code: `axios.post('https://api.example.com/data', payload)`. How can I troubleshoot this issue?"
    - **Suggestion**: Verify the API endpoint URL, check if the API requires authentication or specific headers, and ensure that the `payload` is in the expected format.

CodiRo
Pro in Code - Pro in Life

How to Ask About Programmatic Bugs:

1. **Provide Context and Code**:
    - **Question**: "I'm encountering an `IndexError` in my Python code when accessing a list element. The code is `element = my_list[5]`, and `my_list` is defined as `my_list = [1, 2, 3]`. Why am I getting this error?"
    - **Suggestion**: Ensure that the index you're accessing is within the bounds of the list.
2. **Describe What You Expect vs. What Happens**:
    - **Question**: "My C++ program is supposed to print 'Hello, World!' but instead, it prints nothing. Here's the code: `std::cout << 'Hello, World!' << std::endl;`. What's wrong?"
    - **Suggestion**: Check for syntax errors, such as using single quotes for strings instead of double quotes.
3. **Ask for Help with Debugging**:
    - **Question**: "I'm having trouble debugging a segmentation fault in my C program. Here's the code snippet that causes the crash: `int* ptr = NULL; *ptr = 10;`. How can I resolve this issue?"
    - **Suggestion**: Check for null pointer dereference and initialize pointers before use.

By structuring your questions in this way, you provide enough context for GPT to give you precise and helpful responses.

More Examples of Well-Constructed Questions:

1. **Handling Null Values**:

- **Question**: "In my Java application, I'm encountering a `NullPointerException` when trying to call a method on an object. The code is `user.getName().toUpperCase()`, but `user` might be null. How can I handle this situation safely?"
- **Suggestion**: Use null checks before calling methods on objects, e.g., `if (user != null) { user.getName().toUpperCase(); }`.

2. **Resolving Variable Scope Issues**:
   - **Question**: "I'm trying to use a variable declared inside a function outside of it, but it's causing an `undefined` error. Here's the code: `function foo() { let x = 10; } console.log(x);`. How do I fix this?"
   - **Suggestion**: Ensure variables are declared in the correct scope, or return the variable from the function if needed.
3. **Correcting Syntax Errors**:
   - **Question**: "My Ruby code isn't running correctly, and I'm seeing a `SyntaxError`. Here's the problematic code: `def my_method() puts 'Hello World end`. What's wrong with this code?"
   - **Suggestion**: Ensure that all syntax rules are followed, including closing quotes and parentheses.
4. **Handling Unexpected Data Types**:
   - **Question**: "In my TypeScript code, I'm getting a type error when passing a string to a function that expects a number. The function is `function calculate(num: number) { return num * 2; }`. How can I resolve this issue?"
   - **Suggestion**: Verify the data types being passed and ensure they match the expected types. Use type assertions or conversions if necessary.
5. **Fixing Array Index Errors**:
   - **Question**: "I'm trying to access an element of an array in JavaScript, but I keep getting `undefined`. Here's the code: `let arr = [1, 2, 3]; console.log(arr[5]);`. Why is this happening?"
   - **Suggestion**: Check if the index is within the bounds of the array. In this case, index `5` is out of bounds.
6. **Understanding Library Functions**:
   - **Question**: "I'm using the `math.js` library in JavaScript and calling `math.sqrt('16')`, but it's not returning the expected result. How should I use this function correctly?"
   - **Suggestion**: Check the documentation for the function and ensure correct data types are used.
7. **Debugging Asynchronous Code**:
   - **Question**: "In my Node.js application, I'm using `async/await`, but I'm not getting the expected result from my `fetchData` function. Here's the code: `async function fetchData() { let result = await someAsyncOperation(); console.log(result); }`. How can I troubleshoot this?"
   - **Suggestion**: Ensure that `someAsyncOperation` is returning a promise and handle errors with try-catch.
8. **Fixing Logical Errors**:
   - **Question**: "My algorithm for finding the maximum value in an array is not working correctly. Here's the code: `function findMax(arr) { let max = arr[0]; for (let i = 1; i < arr.length; i++) { if (arr[i] < max) { max = arr[i]; } } return max; }`. What's wrong?"
   - **Suggestion**: Ensure the comparison operator is correct (< should be > to find the maximum).
9. **Understanding Memory Leaks**:
   - **Question**: "My Python application is consuming more memory over time. I suspect a memory leak. Here's a part of my code: `def process_data(data): data.append(new_item)`. How can I identify and fix memory leaks?"
   - **Suggestion**: Check if data is being unintentionally retained or if there are circular references. Use profiling tools to track memory usage.

CodliRo
Pro in Code = Pro in Life

10. **Addressing Performance Issues**:
    - **Question**: "My SQL query is running very slowly. Here's the query: `SELECT * FROM users WHERE age > 30`. How can I optimize this query for better performance?"
    - **Suggestion**: Consider adding indexes to the columns used in the `WHERE` clause and optimizing the database schema.

Additional Suggestions for Asking Programming Questions:

1. **Include Example Inputs and Outputs**:
    - **Question**: "My Python function is not producing the correct output for the given input. Here's the code: `def add_numbers(a, b): return a + b`. If I call `add_numbers(2, 3)`, I expect `5`, but it's giving a different result. What's the issue?"
    - **Suggestion**: Include both the expected and actual outputs for better diagnosis.
2. **Explain the Expected Behavior**:
    - **Question**: "I expect my JavaScript function to update the DOM element with ID `result` with the text 'Done'. Here's the code: `document.getElementById('result').innerText = 'Done';`. It doesn't seem to work. How can I fix this?"
    - **Suggestion**: Verify that the DOM element with ID `result` exists and is correctly targeted.
3. **Ask for Best Practices**:
    - **Question**: "I'm implementing a logging feature in my Node.js application. What are the best practices for managing and rotating log files?"
    - **Suggestion**: Look into logging libraries like `winston` or `morgan` and consider using tools for log rotation.
4. **Seek Alternatives or Improvements**:
    - **Question**: "I'm using a simple recursive algorithm for calculating Fibonacci numbers, but it's slow. Can you suggest a more efficient approach?"
    - **Suggestion**: Consider iterative methods or dynamic programming approaches for better performance.

CodliRo
Pro in Code - Pro in Life

1. Provide Context and Code

Question Construction:

- **Start with a brief description** of what you're trying to achieve or the feature you're implementing.
- **Share the specific code** where the issue is occurring.
- **Describe the problem** clearly, including any error messages or unexpected behavior.

Example:

- **Description**: "I'm developing a user authentication feature in my Python Flask application."
- **Code**:



- **Problem**: "When I attempt to log in with valid credentials, I receive a 401 error instead of the success message. How can I fix this?"

2. Explain Expected vs. Actual Behavior

Question Construction:

- **Clearly state what the code is expected to do**.
- **Describe what is actually happening**, including any discrepancies or errors.

Example:

- **Expected Behavior**: "The `calculateDiscount` function should return the discounted price when given a price and a discount percentage."
- **Code**:

<javascript>

```javascript
function calculateDiscount(price, discount) {
    return price * discount / 100;
}
```

Actual Behavior: "The function returns a discount value of `0` when called with `calculateDiscount(100, 20)`. What's causing this issue?"

3. Include Error Messages and Logs

Pro in Code - Pro in Life

Question Construction:

- **Provide any error messages or stack traces** that are relevant.
- **Include logs** if they help diagnose the problem.

Example:

- **Error Message**: "I'm getting an `IndexError: list index out of range` when running the following code."
- **Code**:

<python>

```python
numbers = [1, 2, 3]
print(numbers[5])
```

Logs/Error Message: "The error traceback indicates the problem is at line 2."

4. Describe What You've Tried

Question Construction:

- **List any debugging steps or fixes** you have already attempted.
- **Mention why these steps did not work** if applicable.

Example:

- **What I Tried**: "I've verified the file path and tried using `os.path.exists()` to check if the file exists, but it still returns a `FileNotFoundError`."
- **Code**:



5. Ask for Best Practices or Alternatives

Question Construction:

- **Request suggestions for best practices** if you're unsure about the implementation.
- **Ask for alternatives** if the current approach isn't working.

Example:

CodliRo
Pro in Code - Pro in Life

- **Request**: "I'm implementing a caching mechanism in my Node.js app but unsure of the best practices. Could you suggest effective strategies for managing cache in a high-load environment?"

Additional Examples Based on Customer-Provided Code Issues

1. **Problem with Code Execution**:
   - **Question**: "In my C++ application, the following code is not behaving as expected. The `printArray` function should output the elements of the array, but it prints nothing. Here's the code:"



   - **Problem**: "The loop condition `i <= size` is causing an out-of-bounds access. How should I correct the loop to avoid this issue?"

2. **Unexpected Output**:
    - **Question**: "I'm working with a JavaScript function that is supposed to filter out even numbers from an array. However, it's not filtering as expected. Here's the function:"

```javascript
function filterEvenNumbers(arr) {
    return arr.filter(num => num % 2 = 0);  // Issue here
}
```

    - **Problem**: "The function is returning incorrect results because of a syntax error. How can I fix this?"
3. **Dependency Issue**:
    - **Question**: "I'm having trouble with a Python script that uses the `requests` library. It fails with an `ImportError`. Here's the relevant part of the code:"

```python
import requests
response =
requests.get('https://api.example.com/data')
```

    - **Problem**: "The script fails with `ImportError: No module named 'requests'`. I've tried reinstalling the library using `pip`, but the issue persists. What else can I try?"
4. **Performance Concern**:
    - **Question**: "My SQL query to retrieve user data is taking too long. Here's the query:"

```sql
SELECT * FROM users WHERE age > 30 ORDER BY name;
```

    - **Problem**: "The query performance is poor. What optimizations can I apply to improve the execution time?"

6. Handling Dependencies and Imports

Question Construction:

- **State the problem with dependencies** or imports.
- **Include code snippets** related to the import or dependency issue.
- **Describe any error messages** or issues experienced.

Example:

- **Description**: "I'm using a third-party library in my Python script, but I'm encountering import errors."
- **Code**:

```python
from some_library import some_function
```

- **Error Message**: "ImportError: cannot import name 'some_function' from 'some_library'."
- **Problem**: "I've installed the library via `pip`, but the import is failing. How can I resolve this?"

7. Issues with Asynchronous Code

Question Construction:

- **Explain the purpose of the asynchronous code** and what it should do.
- **Include the relevant code** and specify where the problem occurs.
- **Detail any issues** or unexpected behavior.

Example:

- **Description**: "I'm working with asynchronous functions in Node.js, but the function does not return the expected result."
- **Code**:

```javascript
async function fetchData() {
    let response = await
axios.get('https://api.example.com/data');
    return response.data;
}
fetchData().then(data => console.log(data));
```

- **Problem**: "The function returns `undefined`. I'm not sure why the data is not being logged. What could be wrong?"

8. Debugging UI Issues

Question Construction:

- **Describe the user interface problem** and what should be happening.
- **Provide the relevant code** for UI components or styles.
- **Include a description** of the issue or bug.

Example:

- **Description**: "I'm having trouble with CSS styling in my HTML page. The button is not displaying correctly."
- **Code**:



- **Problem**: "The button background color is not appearing as expected. It remains grey. How can I fix this?"

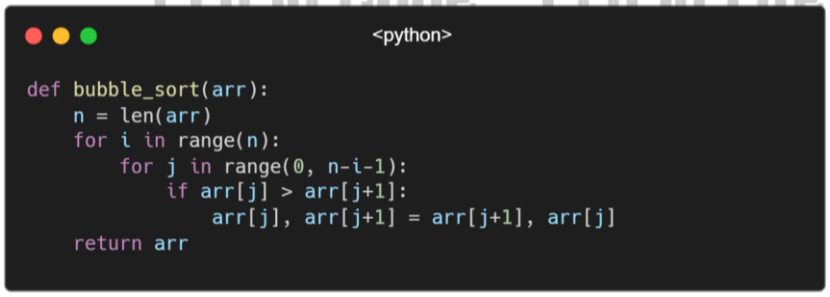
9. Fixing Logic Errors in Algorithms

Question Construction:

- **Explain the algorithm's purpose** and what it's supposed to achieve.
- **Include the code** with the logic error.
- **Describe what's not working** as expected.

Example:

- **Description**: "I'm implementing a sorting algorithm, but it's not sorting the array correctly."
- **Code**:



- **Problem**: "The function does not sort the array in ascending order as expected. What could be causing the issue?"

10. Addressing API Integration Issues

Question Construction:

- **Describe the API integration issue** and what you are trying to accomplish.
- **Include the relevant code** for making the API call.
- **Provide any error messages** or unexpected responses.

Example:

- **Description**: "I'm integrating with an external API, but my `POST` request is returning a `403 Forbidden` error."
- **Code**:

```<javascript>
axios.post('https://api.example.com/submit', { data: 'test' })
  .then(response => console.log(response))
  .catch(error => console.error(error));
```

- **Problem**: "The API call returns a `403 Forbidden` error. How can I resolve this issue?"

11. Troubleshooting Deployment Issues

Question Construction:

- **Explain the deployment issue** and what you're trying to deploy.
- **Include the relevant configuration files** or deployment scripts.
- **Describe the problem** or errors encountered during deployment.

Example:

- **Description**: "I'm having trouble deploying my Node.js application to Heroku."
- **Code**:

Pro in Code - Pro in Life

```<bash>
# Heroku deployment command
git push heroku main
```

- **Problem**: "Deployment fails with an error message: `Application Error`. How can I fix this?"

12. Handling Version Control Issues

Question Construction:

- **Describe the version control issue** and what you're trying to accomplish.
- **Include the relevant commands** or issues with version control.
- **Mention any error messages** or unexpected outcomes.

Example:

- **Description**: "I'm having trouble with merging branches in Git."
- **Code**:

```bash
git checkout feature-branch
git merge main
```

- **Problem**: "I'm encountering merge conflicts. How should I resolve these conflicts and proceed with the merge?"

13. Optimizing Code Performance

Question Construction:

- **State the performance issue** and what the code is supposed to do.
- **Include the relevant code** and where the performance lag is noticed.
- **Describe any profiling results** or performance metrics if available.

Example:

- **Description**: "My Python script for processing large datasets is running slowly."
- **Code**:

```python
def process_data(data):
    results = []
    for item in data:
        results.append(complex_computation(item))
    return results
```

- **Problem**: "The function is slow with large datasets. What are some ways to optimize this code?"

14. Understanding Language-Specific Features

Question Construction:

- **Describe the feature or functionality** you are trying to use.
- **Include the relevant code** that uses this feature.
- **Explain what's not working** as expected.

Example:

- **Description**: "I'm trying to use the `optional chaining` feature in JavaScript."

- **Code**:

```javascript
let user = { profile: { name: 'Alice' } };
console.log(user.profile?.name);
```

- **Problem**: "I’m not sure if optional chaining is working correctly in my environment. How can I verify that it’s supported?"

CodliRo
Pro in Code - Pro in Life